

Frage: Performanz im Stream

```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3, 5, 4, 7, 8, 9);
```

```
    // Variante 1: Zwei Filter
```

```
    result = numbers.stream()  
        .filter(e -> e > 3)  
        .filter(e -> e % 2 == 0)  
        .map(e -> e * 2)  
        .findFirst()  
        .orElse(0);  
    System.out.println(result);
```

```
    // Variante 2: Ein Filter
```

```
    result = numbers.stream()  
        .filter(e > 3 && e % 2 == 0)  
        .map(e -> e * 2)  
        .findFirst()  
        .orElse(0);  
    System.out.println(result);  
}
```

Jede Operation in einer Stream-Pipeline (wie `.filter()`, `.map()`, etc.) fügt eine kleine Schicht Overhead hinzu.

Variante 1 (Zwei Filter):

1. Ein Element wird aus der Liste genommen.
2. Es wird an die Lambda-Funktion von `filter(e > 3)` übergeben.
3. *Wenn* es besteht, wird es an die Lambda-Funktion von `filter(e % 2 == 0)` übergeben.
4. *Wenn* es besteht, wird es an die Lambda-Funktion von `.map()` übergeben.
5. Hier finden potenziell **zwei separate Funktionsaufrufe** für die Filterung statt..

Frage: Performanz im Stream

```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3, 5, 4, 7, 8, 9);
```

```
// Variante 1: Zwei Filter
```

```
result = numbers.stream()  
    .filter(e -> e > 3)  
    .filter(e -> e % 2 == 0)  
    .map(e -> e * 2)  
    .findFirst()  
    .orElse(0);  
System.out.println(result);
```

```
// Variante 2: Ein Filter
```

```
result = numbers.stream()  
    .filter(e > 3 && e % 2 == 0)  
    .map(e -> e * 2)  
    .findFirst()  
    .orElse(0);  
System.out.println(result);  
}
```

Variante 2 (Ein Filter):

1. Ein Element wird aus der Liste genommen.
2. Es wird an die Lambda-Funktion von `filter(e > 3 && e % 2 == 0)` übergeben.
3. *Wenn* es besteht (wobei `&&` selbst "short-circuiting" nutzt), wird es an `.map()` übergeben.
4. Hier findet nur **ein einziger Funktionsaufruf** für die Filterung statt.

Das Zusammenfassen der Bedingungen in einem einzigen `filter` reduziert den Overhead, der durch die Verkettung mehrerer Pipeline-Operationen entsteht. Es ist effizienter, *eine* Lambda-Funktion aufzurufen, die `A && B` prüft, als *zwei* separate Lambda-Funktionen nacheinander aufzurufen

```
public static void main(String[] args) {
    var numbers = List.of(1, 2, 3, 5, 4, 7, 8, 9);
```

```
// Variante 1: Zwei Filter
```

```
result = numbers.stream()
    .filter(e -> e > 3)
    .filter(e -> e % 2 == 0)
    .map(e -> e * 2)
    .findFirst()
    .orElse(0);
System.out.println(result);
```

```
// Variante 2: Ein Filter
```

```
result = numbers.stream()
    .filter(e > 3 && e % 2 == 0)
    .map(e -> e * 2)
    .findFirst()
    .orElse(0);
System.out.println(result);
}
```

Der Unterschied ist vernachlässigbar gering.

In Ihrem spezifischen Beispiel ist der Performance-Unterschied praktisch **nicht messbar** (wir sprechen von Nanosekunden, wenn überhaupt). Das hat zwei Hauptgründe:

Short-Circuiting (FindFirst): Der wichtigste Faktor hier ist `.findFirst()`. Dies ist eine *kurzschließende (short-circuiting)* Operation. Der Stream stoppt die Verarbeitung, sobald das erste passende Element gefunden wurde.

Ablauf (beide Varianten):

Element 1: filter schlägt fehl.

Element 2: filter schlägt fehl.

Element 3: filter schlägt fehl.

Element 5: filter(e > 3) (true) -> filter(e % 2 == 0) (false).

Element 4: filter(e > 3) (true) -> filter(e % 2 == 0) (true) -> map(4 * 2 = 8).

`.findFirst()` hat das Element 8 gefunden. **Der Stream stoppt.**

Die Elemente 7, 8 und 9 werden gar nicht erst angesehen. Da die Verarbeitung so schnell abbricht, fällt der minimale Overhead der doppelten Filterung nicht ins Gewicht.

Exkurs

JIT-Kompilierung (Just-in-Time): Moderne Java Virtual Machines (JVMs) sind extrem intelligent. Der JIT-Compiler wird höchstwahrscheinlich erkennen, dass die beiden aufeinanderfolgenden filter-Operationen in Variante 1 zu einer einzigen zusammengefasst werden können (ein Prozess, der "Loop Fusing" oder "Operator Fusion" ähnelt). Am Ende wird der kompilierte Maschinencode für beide Varianten wahrscheinlich fast identisch sein.

Empfehlung

Obwohl **Variante 2** (ein Filter) technisch gesehen die effizientere Schreibweise ist, ist der Performance-Unterschied in diesem Szenario (und in den meisten realen Szenarien) so minimal, dass er irrelevant ist.

Empfehlung: Wählen Sie die Variante, die lesbarer ist. In diesem Fall ist `filter(e > 3 && e % 2 == 0)` sowohl die performanteste als auch die (vermutlich) lesbarste Option.